

Contents

1 最短路徑

1.1 basic 1

2 圖論

2.1 凸包問題 1

3 數學

3.1 公式 2

1 最短路徑

1.1 basic

```
1 // c++ code
2 #include <bits/stdc++.h>
3 using namespace std;
4
5 int main() {
6     // test comment
7     cout << "test string\n";
8 }
```

2 圖論

2.1 凸包問題

```
1 struct node{
2     int x,y;
3     bool operator<(const node &other) const{
4         return (x<other.x) || (x==other.x &&
5             y<other.y);
6     };
7 };
8 struct ConvexHull{
9     vector<node> vec;
10    int n;
11    void init(vector<node> &v){
12        vec=v;
13        n=v.size();
14    }
15
16    int cross(node a,node o,node b){
17        return
18            (a.x-o.x)*(b.y-o.y)-(b.x-o.x)*(a.y-o.y);
19    }
20
21    vector<int> point;
22    vector<int> area;
23
24    void build(){
25        point=vector<int>(n+1);
26        area=vector<int>(n+1);
27        vector<node> up,down;
28        int upsum=0;
29        int downsum=0;
30        for(int i=0;i<n;i++){
31            while(up.size()>=2 &&
32                cross(up[up.size()-2],up[up.size()-1],vec[i])<=0)
33                up.pop_back();
34            up.push_back(vec[i]);
35            if(up.size()>1)
36                upsum+=cross(up[up.size()-2],{0,0},up[up.size()-1]);
37
38            while(down.size()>=2 &&
39                cross(down[down.size()-2],down[down.size()-1],vec[i])>=0)
40                down.pop_back();
41            down.push_back(vec[i]);
42            if(down.size()>1)
43                downsum+=cross(down[down.size()-2],{0,0},down[down.size()-1]);
44
45            area[i+1]=abs(upsum-downsum);
46
47            if(i+1==1) point[i+1]=1;
48            else point[i+1]=up.size()+down.size()-2;
49        }
```

```
39        downsum-=cross(down[down.size()-2],{0,0},down[down.size()-1]);
40        down.pop_back();
41    }
42    down.push_back(vec[i]);
43    if(down.size()>1)
44        downsum+=cross(down[down.size()-2],{0,0},down[down.size()-1]);
45
46    area[i+1]=abs(upsum-downsum);
47
48    if(i+1==1) point[i+1]=1;
49    else point[i+1]=up.size()+down.size()-2;
50 }
51
52 double getarea(int p){
53     return area[p]/2.0;
54 }
55
56 int getnum(int p){
57     return point[p];
58 }
59 };
60
61 struct node{
62     int x,y;
63     bool operator<(const node &other) const{
64         return (x<other.x) || (x==other.x &&
65             y<other.y);
66     };
67 };
68
69 struct ConvexHull{
70     vector<node> vec;
71     int n;
72     void init(vector<node> &v){
73         vec=v;
74         n=v.size();
75     }
76
77     int cross(node a,node o,node b){
78         return
79             (a.x-o.x)*(b.y-o.y)-(b.x-o.x)*(a.y-o.y);
80     }
81
82     vector<int> point;
83     vector<int> area;
84
85     void build(){
86         point=vector<int>(n+1);
87         area=vector<int>(n+1);
88         vector<node> up,down;
89         int upsum=0;
90         int downsum=0;
91         for(int i=0;i<n;i++){
92             while(up.size()>=2 &&
93                 cross(up[up.size()-2],up[up.size()-1],vec[i])<=0)
94                 upsum-=cross(up[up.size()-2],{0,0},up[up.size()-1]);
95                 up.pop_back();
96             up.push_back(vec[i]);
97             if(up.size()>1)
98                 upsum+=cross(up[up.size()-2],{0,0},up[up.size()-1]);
99
100            while(down.size()>=2 &&
101                cross(down[down.size()-2],down[down.size()-1],vec[i])>=0)
102                downsum-=cross(down[down.size()-2],{0,0},down[down.size()-1]);
103                down.pop_back();
104            down.push_back(vec[i]);
105            if(down.size()>1)
106                downsum+=cross(down[down.size()-2],{0,0},down[down.size()-1]);
107
108            area[i+1]=abs(upsum-downsum);
109
110            if(i+1==1) point[i+1]=1;
111            else point[i+1]=up.size()+down.size()-2;
112        }
```

```
50 |     }  
51 |  
52 |     double getarea(int p){  
53 |         return area[p]/2.0;  
54 |     }  
55 |  
56 |     int getnum(int p){  
57 |         return point[p];  
58 |     }  
59 | };
```

3 數學

3.1 公式

- 中文測試

- $\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$